Question 1 of 30

Care din urmatoarele afirmatii sunt adevarate?

Correct answer: **A) C) D)**
You chose: **A) D)**
Note: This question displayed answers in random order during the test.

- ✔ **A)** Un destructor poate fi declarat virtual.
- **B)** Un constructor poate fi declarat virtual.
- ➔ **C)** Daca am declarat o metoda virtuala in clasa de baza atunci ea va fi virtuala si in clasele derivate.
- ✔ **D)** Putem opri suprascrierea unei metode virtuale in clasele derivate adaugand cuvantul cheie final la declaratie metodei.
- **E)** Daca un constructor este declarat ca virtual, atunci si destructorul trebuie declarat ca virtual.

You did not select all available correct options.

Points: 0.7 out of 1

Question 2 of 30

Se poate defini o variabila (data membru) private, utilizand atat specificatorul **const** cat si **mutable** ?

Correct answer: **A)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

- ✔ **A)** Da, cu conditia ca cei doi specificatori sa nu se refere la acelasi lucru (ex: unul se refera la pointer, altul la valoarea lui)
- **B)** Da, mutable suprascrie calitatea de const
- **C)** Nu, cei doi specificatori nu pot fi utilizari in acelasi timp
- **D)** Da, dar doar daca variabila este privata si const este utilizat inainte lui mutable
- **E)** Da, dar doar daca mutable e utilizat primul

Points: 2 out of 2

Question 3 of 30

Ce se va afisa in urma executiei codului de mai jos ? (Executia si compilarea se realizeaza pe o masina de 32 de biti).

```
#include <iostream>
using namespace std;

class A {
    int a;
public:
    virtual void run() = 0;
};

class B : public A {
    int a;
public:
    void run();
};

int main()
{
    cout << sizeof(A) << "\n";
    cout << sizeof(A) + sizeof(B) << "\n";
    return 0;
}
```

Correct answer: D)
You chose: D)
Note: This question displayed answers in random order during the test.

A) 4
12

B) 4
8

C) 8
12

✓ **D)** 8
20

E) 8
16

F) 4
4

G) 8
8

H) Eroare la compilare

Points: 1 out of 1

Question 4 of 30

Data urmatoarea clasa template Numar, care din urmatoarele declaratii de supraincarcare a operatorului += sunt corecte, astfel incat sa se poata realiza operatia x+=3, unde x e un obiect de tip Numar<int>;

```
template <class T>
class Numar
{
   T x;
   public:
     Numar(){};
};
```

Correct answer: **A) C)**
You chose: **A) D)**
Note: This question displayed answers in random order during the test.

✔ **A)** void operator+=(const T& y);

B) void operator+=(T& y);

➡ **C)** void operator+=(T y);

✘ **D)** void operator+=(const Numar& y);

Question 5 of 30

Presupunand ca avem o variabila de tipul **int** cu numele "**x**" , care din urmatoarele metode poate fi utilizata pentru a afisa valoarea lui "**x**" pe ecran ?

Correct answer: **A)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

✔ **A)** cout << x;

B) cin >> x;

C) printf("%c",x);

D) scanf("%c",x);

Points: 1 out of 1

Question 6 of 30

Ce afiseaza codul de mai jos?

```
#include <iostream>
using namespace std;


int main()
{
    cout << sizeof(int*) + sizeof(char*) - sizeof(long long int*) - sizeof(unsigned char*) << endl;
    return 0;
}
```

Correct answer: **A)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

✔ **A)** 0

B) -4

C) Codul nu compileaza.

D) -4, daca compilam pe 32 biti si 0, daca compilam pe 64 biti. Deoarece dimensiunea lui int este 8 pe 64 biti si 4 pe 32 biti.

Points: 1 out of 1

Question 7 of 30

Daca avem o functie templetizata care primeste un argument de tipul **const T**, ce tipuri de valori putem folosi ca argumente.

Correct answer: B) C)
You chose: C)
Note: This question displayed answers in random order during the test.

- **A)** Pointeri la instante de tipul T.
- ➡ **B)** Instante de tipul T.
- ✓ **C)** Instante constante de tipul T.
- **D)** Pointeri la instante constante de tipul T.

You did not select all available correct options.

Points: 0.5 out of 1

Question 8 of 30

Care dintre afirmatiile de mai jos sunt adevarate pentru functii friend?

Correct answer: A) B) C) D)
You chose: A) B)
Note: This question displayed answers in random order during the test.

- ✓ **A)** O functie friend a unei clase are acces la membrii private si protected ale acelei clase.
- ✓ **B)** O functie friend a unei clase poate fi definita in afara acelei clase.
- ➡ **C)** O functie friend a unei clase este declarata in acea clasa.
- ➡ **D)** O functie friend a unei clase nu este membru al acelei clase.

You did not select all available correct options.

Points: 0.5 out of 1

Question 9 of 30

O eroare produsă de o excepție netratată, este

Correct answer: C)
You chose: A)
Note: This question displayed answers in random order during the test.

- ✗ **A)** o eroare de compilare
- **B)** o eroare de editare legături (linking)
- ➡ **C)** o eroare ce apare în timpul execuției
- **D)** o eroare logică

Points: 0 out of 1

Question 10 of 30

Ce va fi afisat in urma compilarii si rularii codului?

```
#include <cstdio>
class Cat
{
public:
    Cat() { printf("CAT "); }
};

class Dog
{
public:
    Dog() { printf("DOG "); }
};

class Hamster
{
public:
    Hamster() { printf("HAMSTER "); }
};

class ZOO
{
    Dog dog1,dog2;
    Hamster Hamster;
    Cat cat;
public:
    ZOO() { printf("ZOO "); }
};

int main()
{
    ZOO z;
    return 0;
}
```

Correct answer: **A)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

✓ **A)** DOG DOG HAMSTER CAT ZOO

B) ZOO DOG DOG HAMSTER CAT

C) ZOO CAT HAMSTER DOG DOG

D) CAT HAMSTER DOG DOG ZOO

E) Nu compileaza deci nu se va afisa nimic

Points: 1 out of 1

Question 11 of 30

Ce afiseaza codul de mai jos?

```
#define CHECK_IF_DIGIT(x) {                 \
    if (x > 9 || x < 0) {                   \
        printf("%d is not a digit!\n", x);  \
    }                                       \
}
int main() {
    int x = 10;
    CHECK_IF_DIGIT(x++);
    return 0;
}
```

Correct answer: **B)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

- ✕ **A)** 10 is not a digit!
- ➜ **B)** 11 is not a digit!
- **C)** 12 is not a digit!
- **D)** 13 is not a digit!

Points: 0 out of 2

Question 12 of 30

Ce se va afisa in urma executiei codului de mai jos? Codul ASCII pentru 'a' este 97.

```
#include <iostream>
using namespace std;

template<class T>
class Sum {
    int sum;
public:
    Sum() {
        sum = 0;
    }
    void Add(T x) { sum += x; };
    void Add(int x) { sum -= x; };
    void Print() { cout << sum; }
};

void main()
{
    Sum<char> s;
    s.Add(5);
    s.Add(true);
    s.Add('a');
    s.Print();
}
```

Correct answer: B)
You chose: C)
Note: This question displayed answers in random order during the test.

- **A)** 92
- **B)** 91
- ✗ **C)** 93
- **D)** Eroare la compilare

Question 13 of 30

Ce operatori **NU** pot fi supraincarcati?

Correct answer: D) H)
You chose: C) D) H)
Note: This question displayed answers in random order during the test.

- **A)** new
- **B)** delete
- ✗ **C)** . (operator punct)
- ✓ **D)** ::
- **E)** , (operator virgula)
- **F)** []
- **G)** ()
- ✓ **H)** sizeof

You did not select all available correct options.

Points: 0.5 out of 1

Question 14 of 30

Data fiind clasa Numar de mai jos, care este declaratia corecta a operatorului + pentru a permite operatia b=3+a, unde b si a sunt obiecte de tip Numar;

```
class Numar
{
    int nr;
public:
    Numar();
}
```

Correct answer: **D)**
You chose: **B) D)**
Note: This question displayed answers in random order during the test.

A)

```
Numar operator+(const Numar& x);
```

✗ **B)**

```
Numar operator+(int x);
```

C)

```
friend Numar operator+(const Numar& x, int y);
```

✓ **D)**

```
friend Numar operator+(int x, const Numar& y);
```

You did not select all available correct options.

Question 15 of 30

Ce va face urmatorul cod?

```
#include <iostream>
using namespace std;

class Parent {
    int foo;
public:
    Parent() : foo(69) {}
    virtual int bar()
    {
        return foo;
    }
};


class Child: public Parent {
public:
    int bar() {
        return (foo + 2);
    }
};


int main()
{
    Parent* p = new Child;
    cout << p->bar() << endl;
    return 0;
}
```

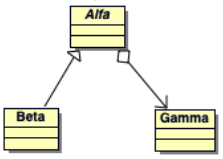
Correct answer: **C)**
You chose: **B)**
Note: This question displayed answers in random order during the test.

- **A)** Va compila si va afisa 69 pentru ca 'p' e un pointer de tip Parent* si se apeleaza metoda 'bar' din clasa Parent.
- ✗ **B)** Va compila si va afisa 71 pentru ca, desi pointerul 'p' este de tip Parent*, el are adresa unui obiect de tip Child. Asadar se apeleaza metoda 'bar' din clasa Child.
- ➜ **C)** Codul nu va compila.
- **D)** Codul va compila, dar nu va afisa nimic.

Points: 0 out of 2

Question 16 of 30

Care afirmații sunt adevărate despre diagrama de mai jos?



Correct answer: **A) D) F)**
You chose: **A) B) E) F)**
Note: This question displayed answers in random order during the test.

- ✓ **A)** Clasa Alfa este abstractă.
- ✗ **B)** Clasa Beta este abstractă.
- **C)** Între clasa Alfa și clasa Beta există o relație de asociere.
- ➜ **D)** Între clasa Alfa și clasa Beta există o relație de generalizare/specializare.
- ✗ **E)** Un obiect Alfa are in componența sa un obiect Gamma.
- ✓ **F)** Un obiect din ierarhia Alfa are in componența sa un obiect Gamma.
- **G)** Diagrama este greșită deoarece nu poate exista relație de agregare între o clasă abstractă și una concretă.
- **H)** Relația de agregare spune că un obiect Gamma are în componența sa un obiect din ierarhia Alfa.

Question 17 of 30

Fie metoda *Filter* din clasa templetizata *MyVector* declarata in felul urmator.

```
template <typename T>
class MyVector
```

Aceasta primeste ca si parametru o functie de filtrare care proceseaza un element de tipul template-ului utilizat in MyVector si returneaza *true/false*. Care este definitia corecta pentru metoda *Filter*?

Correct answer: **D)**
You chose: **D)**
Note: This question displayed answers in random order during the test.

- **A)** void Filter(bool filter_func(int e));
- **B)** void Filter(bool(*filter_func)(int e));
- **C)** bool Filter((bool*)filter_func(T e));
- ✓ **D)** void Filter(bool(*filter_func)(T e));

Points: 1 out of 1

Question 18 of 30

Ce va afișa execuția următorului cod (se considera ca include-urile necesare pentru ca acest program sa ruleze sunt adaugate):

```
void f() { throw "Hello "; }
void g() { throw "World!"; }
int main()
{
  try {
    try {
      f();
    }
    catch(char* msg) {
      std:: cout << msg;
    }
    g();
  }
  catch(char* msg) {
    std:: cout << msg << "\n";
  }
  return 0;
}
```

Correct answer: **D)**
You chose: **C)**
Note: This question displayed answers in random order during the test.

A) Hello

B) World!

✕ **C)** Hello World!

➜ **D)** un mesaj de eroare de tipul "...terminating with uncaught exception of type..."

Question 19 of 30

Ce va afisa codul de mai jos?

```
class Foo {
    int count;
    int v[100];
public:
    Foo() {
        count = 5;
        for (int i = 0; i < count; ++i) {
            v[i] = i;
        }
    }
    Foo(initializer_list<int> list) {
        count = 0;
        for (auto x : list) {
            v[count++] = x;
        }
    }
    void Print() {
        for (int i = 0; i < count; ++i) {
            cout << v[i] << " ";
        }
    }
};
int main() {
    Foo f { };
    f.Print();

    return 0;
}
```

Correct answer: **B)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

✕ **A)** Nimic, deoarece am folosit o lista de initializare goala.

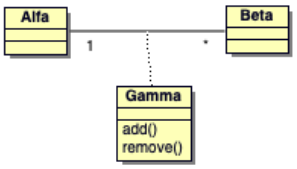
➜ **B)** 0 1 2 3 4

C) Eroare de compilare.

D) Eroare la rulare.

Question 20 of 30

Care afirmații sunt adevărate despre diagrama de mai jos?



Correct answer: C) F)
You chose: C) F)
Note: This question displayed answers in random order during the test.

A) La implementare, clasa Alfa include o dată membru ce referă obiecte Beta.

B) La implementare, clasa Beta include o dată membru ce referă un obiect Alfa.

✔ **C)** Metoda add() este responsabilă cu adăugarea unei relații de asociere dintre un obiect Alfa și obiecte Beta.

D) Metoda add() este responsabilă cu adăugarea unei relații de asociere dintre un obiect Alfa și un obiect Gamma.

E) Un obiect Gamma este compus din un obiect Alfa și mai multe obiecte Beta.

✔ **F)** Metoda remove() elimină o asociere dintre un obiect Alfa si obiecte Beta.

G) Metoda remove() elimină o asociere dintre un obiect Alfa si un obiect Beta.

Question 21 of 30

Ce se va afisa la executia codului de mai jos?

```cpp
#include <iostream>

class A {
public:
    virtual void f() { std::cout << "A"; }
};

class B : public A {
public:
    void f() { std::cout << "B"; }
};

void g(A &a) { a.f(); }

int main() {
    B b;
    g(b);
}
```

Correct answer: B)
You chose: C)
Note: This question displayed answers in random order during the test.

A) A

B) B

C) Codul nu compileaza

D) AB

Question 22 of 30

Polimorfismul la runtime se realizeaza prin:

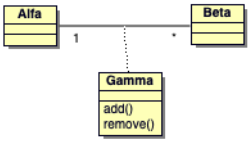
Correct answer: D)
You chose: D)
Note: This question displayed answers in random order during the test.

- **A)** Supraincarcarea functiilor
- **B)** Suprascrierea functiilor
- **C)** Clase template
- ✓ **D)** Functii virtuale
- **E)** Functii friend

Points: 1 out of 1

Question 23 of 30

Care afirmații sunt adevărate despre diagrama de mai jos?



Correct answer: C) E) F) H)
You chose: B) E) G)
Note: This question displayed answers in random order during the test.

- **A)** Este o relație de asociere ternară.
- ✗ **B)** Un obiect Alfa este asociat cu un obiect Beta.
- ➡ **C)** Un obiect Alfa poate fi asociat cu obiecte Beta.
- **D)** Un obiect Beta poate fi asociat cu obiecte Gamma.
- ✓ **E)** Un obiect Beta este asociat cu un obiect Alfa.
- ➡ **F)** Un obiect Gamma gestionează relațiile de asociere dintre obiectele Alfa și Beta.
- ✗ **G)** Un obiect Gamma este responsabil cu adăugarea și ștergerea de obiecte Alfa și Beta.
- ➡ **H)** Clasa Gamma ar putea fi implementată utilizând șablonul Singleton.

Question 24 of 30

Cati destructori poate avea o clasa ?

Correct answer: **A) D)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

- ✓ **A)** 1
- **B)** 2
- **C)** Oricat de multi
- ➜ **D)** nici unul

You did not select all available correct options.

Points: 0.5 out of 1

Question 25 of 30

Cum putem crea o clasa, in care sa declaram un singur constructor, care sa nu fie abstracta (sa nu contina nici o metoda pura virtuala) si pentru care nu se pot instantia obiecte:

Correct answer: **A) B)**
You chose: **A)**
Note: This question displayed answers in random order during the test.

- ✓ **A)** Facem acel constructor privat
- ➜ **B)** utilizam cuvantul cheie **delete** pentru a sterge acel constructor
- **C)** facem acel constructor virtual (=0)
- **D)** facem constructorul public, dar adauga specificatorul **explicit** in definitia lui.

You did not select all available correct options.

Question 26 of 30

Care din urmatoarele raspunsuri reflecta un polimorfism static ?

Correct answer: **A)**
You chose: **D)**
Note: This question displayed answers in random order during the test.

A)

```
template <typename T> class A { ... }
class B : public A<B> { ... }
```

B)

```
class A { ... }
template <typename T> class B { ... }
class B : public A { ... }
```

C)

```
class A {
virtual void foo() = 0;
}
class B: public A {
virtual void foo() override { ... }
}
```

D)

```
class A {
void foo() = 0;
}
class B: public A {
void foo() override { ... }
}
```

Question 27 of 30

Care dintre afirmatii sunt adevarate pentru codul de mai jos?

```
#include <iostream>
using namespace std;


class Foo {
    int val;
public:
    Foo(): val(69) { }

    Foo& operator--()
    {
        Foo aux;
        aux.val = val - 1;
        return aux;
    }
};


int main()
{
    Foo bar;
    bar--;
    return 0;
}
```

Correct answer: B) C)
You chose: B) C)
Note: This question displayed answers in random order during the test.

A) Codul compileaza si ruleaza.

✓ **B)** Codul nu compileaza pentru ca nu a fost suprascris operatorul postdecrementare in clasa Foo.

✓ **C)** In definitia supraincarcarii operatorului se returneaza referinta la o variabila locala si acest lucru poate duce la comportament nedefinit.

D) Supraincarcarea operatorului este gresita deoarece nu este permis un tip de return diferit de 'void' pentru supraincarcarea operatorilor de incrementare si decrementare.

Question 28 of 30

```
#include <iostream>

using namespace std;

long double operator"" _km(unsigned int x)
{
    return x * 1000.0;
}
long double operator"" _m(unsigned int x)
{
    return x;
}
int main()
{
    long double distance = 420_km;
    distance += 43_m;
    cout << distance;
}
```

Ce se va întampla în urma compilării și rulării codului:

Correct answer: B)
You chose: B)
Note: This question displayed answers in random order during the test.

A) 420043

✔ **B)** Codul nu compileaza

C) 463

D) 42043

Question 29 of 30

Ce se va afisa in urma executiei codului de mai jos?

```
#include <iostream>
using namespace std;

int main() {
    int x[5] = {-1, 2.0, 3.5, 4.4, 5.6};
    for (auto i : x) {
        cout << i * 2 << " ";
    }
    return 0;
}
```

Correct answer: **A)**
You chose: **E)**
Note: This question displayed answers in random order during the test.

- ➡ **A)** -2 4 6 8 10
- **B)** Eroare de compilare
- **C)** -2.0 4.0 7.0 8.8 11.2
- **D)** -2 4 7 8.8 11.2
- ✕ **E)** -2 4 7 9 11
- **F)** -2 4 7 8 11

Question 30 of 30

Care este definita corecta pentru un constructor de copiere pentru clasa Point de mai jos?

```
class Point {
    int x, y;
public:
    Point() {
        x = 0; y = 0;
    }
};
```

Correct answer: **B)**
You chose: **C)**
Note: This question displayed answers in random order during the test.

- **A)** Point(int x1, int y1) { x = x1; y = y1; }
- ➡ **B)** Point(const Point &p2) {x = p2.x; y = p2.y; }
- ✕ **C)** Point(const Point &&p2) {x = p2.x; y = p2.y; }
- **D)** Point(Point* p2) {x = p2->x; y = p2->y; }